EXPERIMENT NO 1

```
from qiskit import *
from qiskit_aer import *
from qiskit.visualization import *
from qiskit.quantum_info import *
qc = QuantumCircuit(1, 1)
qc.x(0)
qc.z(0)
qc.h(0)
print(qc)
qc.measure(0, 0)
simulator = AerSimulator()
result = simulator.run(qc, shots=1024).result()
counts = result.get_counts()
print("\nMeasurement Counts:")
print(counts)
plot_histogram(counts)
```

EXPERIMENT NO 2

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
from qiskit.quantum_info import Statevector
qc = QuantumCircuit(2, 2)
qc.x(1)
qc.h(0)
qc.cx(0, 1)
print(qc)
state = Statevector.from_instruction(qc)
print("\nStatevector:")
print(state)
print("\nProbabilities:")
print(state.probabilities_dict())
qc.measure([0, 1], [0, 1])
simulator = AerSimulator()
result = simulator.run(qc, shots=1024).result()
counts = result.get_counts()
print("\nMeasurement Counts:")
print(counts)
plot_histogram(counts)
```

EXPERIMENT NO 3

```
from qiskit import *
from qiskit_aer import *
from qiskit.visualization import *
from qiskit.quantum_info import *
qc = QuantumCircuit(2, 2)
qc.h(0)
qc.cx(0, 1)
print(qc)
state = Statevector.from_instruction(qc)
print("\nBell State:")
print(state)
print("\nProbabilities:")
print(state.probabilities_dict())
qc.measure([0, 1], [0, 1])
simulator = AerSimulator()
result = simulator.run(qc, shots=1024).result()
counts = result.get_counts()
print("\nMeasurement Counts:")
print(counts)
plot_histogram(counts)
```

EXPERIMENT NO 4

```
from qiskit import *
from qiskit_aer import *
from qiskit.visualization import *
qc = QuantumCircuit(2, 2)
qc.h(0)
qc.cx(0, 1)
message = "11"
if message == "00":
 pass
elif message == "01":
 qc.z(0)
elif message == "10":
 qc.x(0)
elif message == "11":
 qc.z(0)
 qc.x(0)
 qc.cx(0, 1)
qc.h(0)
print(qc)
qc.measure([0, 1], [0, 1])
simulator = AerSimulator()
result = simulator.run(qc, shots=1024).result()
counts = result.get_counts()
print("\nMessage Sent:")
print(message)
print("\nDecoded Counts:")
print(counts)
plot_histogram(counts)
```

EXPERIMENT NO 5

```
from qiskit import *
from qiskit_aer import *
from qiskit.visualization import *
from qiskit.quantum_info import *
qc = QuantumCircuit(2, 2)
qc.x(1)
qc.cx(0, 1)
qc.cx(1, 0)
qc.cx(0, 1)
print(qc)
qc.measure([0, 1], [0, 1])
simulator = AerSimulator()
result = simulator.run(qc, shots=1024).result()
counts = result.get_counts()
print("\nMeasurement Counts:")
print(counts)
plot_histogram(counts)
```

Exp 6 Constant Oracle

```
from qiskit import *
from qiskit_aer import *
from qiskit.visualization import *
from qiskit.quantum_info import *
n = 2
qc = QuantumCircuit(n + 1, n)
qc.x(n)
qc.h(range(n + 1))
qc.h(range(n))
print(qc)
qc.measure(range(n), range(n))
simulator = AerSimulator()
result = simulator.run(qc, shots=1024).result()
counts = result.get_counts()
print("\nMeasurement Counts:")
print(counts)
plot_histogram(counts)
```

Balanced Oracle

```
from qiskit import *
from qiskit_aer import *
from qiskit.visualization import *
from qiskit.quantum_info import *
n = 2
qc = QuantumCircuit(n + 1, n)
qc.x(n)
qc.h(range(n + 1))
for i in range(n):
    qc.cx(i, n)
qc.h(range(n))
print(qc)
qc.measure(range(n), range(n))
simulator = AerSimulator()
result = simulator.run(qc, shots=1024).result()
counts = result.get_counts()
print("\nMeasurement Counts: ")
print(counts)
plot_histogram(counts)
```

Exp 7

```
from qiskit import *
from qiskit_aer import *
from qiskit.visualization import *
from qiskit.quantum_info import *
qc = QuantumCircuit(2, 2)
qc.h([0, 1])
qc.cz(0, 1)
qc.h([0, 1])
qc.x([0, 1])
qc.h(1)
qc.cx(0, 1)
qc.h(1)
qc.x([0, 1])
qc.h([0, 1])
print(qc)
qc.measure([0, 1], [0, 1])
simulator = AerSimulator()
result = simulator.run(qc, shots=1024).result()
counts = result.get_counts()
print("\nMeasurement Counts : "  )
print(counts)
plot_histogram(counts)
```

Exp 8

```
from qiskit import *
from qiskit_aer import *
from qiskit.visualization import *
from qiskit.quantum_info import *
n = 2
qc = QuantumCircuit(n + 1, n)
qc.x(n)
qc.h(range(n + 1))
for i in range(n):
 qc.cx(i, n)
qc.h(range(n))
print(qc)
qc.measure(range(n), range(n))
simulator = AerSimulator()
result = simulator.run(qc, shots=1024).result()
counts = result.get_counts()
print("\nMeasurement Counts:")
print(counts)
plot_histogram(counts)
```